

Security, Operations, and Governance Handbook

A controlled snapshot generated from the governed Markdown source. The live knowledge base remains authoritative for current guidance.

Klar release	2.6.0.1
Classification	PUBLIC
Generated	2026-08-10
Implementation baseline	ccc20090c3ed
KB content hash	2a9943544a7daa14
Controlled-copy notice	Verify against the live KB before operational use.

Table of contents

Governance

3

Klar Knowledge Base Charter

4

Source of Truth

7

Document Lifecycle

10

Documentation Style Guide

13

Documentation Governance

16

Contributing Documentation

20

Security and Privacy

24

Security, Privacy, and Threat Model

25

Quality and Operations

31

Test Strategy and Release Gates

32

Operations Runbooks

38

Known Gaps and Risk Register

43

Decisions and Roadmap

51

Architecture Decisions and Roadmap

52

Governance

6 controlled documents in this section.

Klar Knowledge Base Charter

Mandate, audiences, boundaries, and quality standard for Klar's governed documentation system.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Documentation
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Mandate

The Klar Knowledge Base (KB) is the governed explanation layer for the product. It connects what Klar does, why it does it, how people use it, how it is implemented, how it is operated, and what evidence supports each claim.

The KB exists to reduce operational ambiguity. A reader should be able to identify the current product behavior, the responsible owner, the applicable release, the relevant controls, and the safe next action without reconstructing the answer from source code or old planning material.

Intended outcomes

The KB must enable:

- a new user to complete a safe first session and create a backup;
- support staff to diagnose a problem without requesting sensitive career data;
- product and engineering teams to understand the purpose and boundaries of a capability;
- reviewers to trace a statement to current implementation or approved policy;
- release owners to identify stale, target, deprecated, or blocked documentation;
- auditors and decision-makers to distinguish current evidence from future intent.

Audiences

The public KB serves people who use Klar, help others use it, evaluate it, build it, or review its safeguards. No prior relationship with the project is required to read the published material.

Audience	Primary need	Default reading path
Students and job seekers	Complete a job-search task safely and understand what happens to their data	Getting started and user guides

Audience	Primary need	Default reading path
Schools, universities, and career services	Facilitate safe learning or advising sessions without collecting participant career data	For schools and career services
Developers and technical reviewers	Understand contracts, invariants, architecture, and verification evidence	Architecture, code, data, engineering, and quality sections
Privacy and security reviewers	Evaluate data flows, trust boundaries, controls, limitations, and reporting routes	Security, privacy, data, platform, and governance sections
Product, design, support, and operations	Understand purpose, journeys, operating boundaries, and recovery procedures	Klar overview , user journeys, and runbooks
Contributors, leadership, and independent reviewers	Assess source authority, status, ownership, decisions, and risk	Governance, decisions, and release evidence

Documentation families

The KB contains distinct document families. They must not be blended in a way that obscures authority.

- 1 **Concepts** explain purpose, principles, and domain language.
- 2 **User guides** provide outcome-oriented instructions for current product behavior.
- 3 **Technical reference** records exact contracts, schemas, configuration, and component responsibilities.
- 4 **How-to guides** describe a bounded operational or engineering task.
- 5 **Policies and controls** state mandatory organizational requirements.
- 6 **Runbooks** provide tested response and recovery procedures.
- 7 **Decisions** preserve the context and rationale for consequential choices.
- 8 **Release evidence** records dated proof for a specific build or deployment.
- 9 **Target documents** describe approved future intent and are never presented as current behavior.

Scope and boundaries

The KB documents the authoritative repository state for Klar v2.6.0.1, including the browser/PWA application, Cloudflare Worker boundary, experimental Electron desktop build that is not publicly distributed, local persistence, source connectors, matching, application documents, and release controls.

The KB does not replace:

- executable code, schemas, tests, or configuration;
- legal advice, tax advice, or a formal compliance certification;
- source-provider terms and policies;
- a release decision or security risk acceptance;
- issue tracking or roadmap prioritization.

See [Source of truth](#) for conflict resolution and [Scope and non-goals](#) for product boundaries.

Quality standard

A publishable page must be:

- **accurate:** verified against the applicable implementation and tests;
- **purposeful:** it explains why the subject exists, not only what it is called;
- **bounded:** current, target, historical, and deprecated material are visibly distinct;
- **actionable:** a reader can identify the safe next step;
- **traceable:** ownership, version, verification date, and related evidence are present;
- **privacy-safe:** examples are synthetic and contain no credentials or personal career data;
- **maintainable:** one concept has one canonical page, with links instead of copied passages;
- **accessible:** headings, link text, tables, and language remain usable without visual cues alone.

Success measures

The documentation owner should monitor:

- pages past their review date;
- pages without an active owner;
- broken internal and external links;
- unsuccessful KB searches and repeated support questions;
- release changes without matching documentation changes;
- user procedures that no longer match the interface;
- controls or runbooks without recent test evidence.

Governing documents

- [Source of truth](#)
- [Document lifecycle](#)
- [Documentation style guide](#)
- [Documentation governance](#)
- [Contributing documentation](#)

Source of Truth

Authority hierarchy and conflict-resolution rules for current, target, and historical Klar information.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Documentation
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Governing rule

No single prose page defines Klar's current behavior. Current behavior is established by the released code, configuration, data contracts, and passing verification evidence for the stated release. Documentation explains that evidence and must change when the evidence changes.

Authority hierarchy

Use this order when two sources conflict:

- 1 **Released code, configuration, and data schemas** define what the product can execute.
- 2 **Tests and release evidence** establish which behavior was verified for a release.
- 3 **Release metadata and the changelog** identify the shipped version and historical change.
- 4 **Approved architecture and product decisions** explain why a consequential choice was made.
- 5 **Approved policies and runbooks** define mandatory organizational handling.
- 6 **Current KB pages** explain the verified state for their audience.
- 7 **Roadmaps and target documents** describe intended future behavior only.
- 8 **Old build guides, discussions, and archived documents** are historical context, not current authority.

The Klar application repository that contains this knowledge base is the authoritative implementation source for the KB. Planning material must never be used to reconstruct current behavior when the repository can be inspected directly.

Current, target, and historical statements

Every material claim must fit one of these categories:

Category	Meaning	Required treatment
Current	Present in the applicable released implementation	Verify against code and proportionate tests

Category	Meaning	Required treatment
Target	Approved or proposed future behavior not yet released	Use status: target, or status: planned when it is planning intent rather than an approved product contract; state dependencies and decision state
Historical	True for an earlier release	Name the release and link to history
Deprecated	Still present but scheduled for removal or unsupported use	State replacement and end condition
Unknown	Evidence is absent or contradictory	Do not convert uncertainty into a claim; record a documentation gap

Target content may explain intent but must not use present-tense product language such as “Klar supports” until implementation and verification are complete.

Resolving a conflict

When documentation and implementation differ:

- 1 confirm the applicable Klar version and source commit;
- 2 reproduce the behavior using the supported interface where practical;
- 3 inspect the code, configuration, schema, and relevant tests;
- 4 determine whether the defect is in the product, the documentation, or both;
- 5 correct the responsible source through normal review;
- 6 mark the page as draft or add a visible limitation until the discrepancy is resolved;
- 7 add release evidence when the correction ships.

Do not silently rewrite a page to match an unreviewed local change. An uncommitted or unreleased implementation is not a released product claim.

Evidence expectations

The strength of evidence should match the claim:

- A UI instruction should be verified against the current rendered workflow.
- A data-retention or encryption statement should be verified against storage and cryptographic code plus tests.
- A connector capability should be verified against its registry contract, current route, and contract tests.
- A release-readiness statement should cite the complete release gate and dated results.
- A third-party fact should cite the provider's current primary documentation and carry a verification date.

Absence of a failing test is not proof of a broad guarantee. Use bounded language, especially for generated writing, third-party source coverage, security, accessibility, and compatibility.

Page-level traceability

Every KB page must declare the applicable version, owner, status, classification, verification date, next review date, audience, and tags. Technical pages should also link to relevant components, tests, schemas, controls, decisions, and runbooks.

See [Document lifecycle](#) for status changes and [Documentation governance](#) for ownership and review.

Document Lifecycle

Status model, review stages, triggers, and archival rules for Klar knowledge-base content.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Documentation
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Status model

Status	Use when	Publication rule
draft	Content is incomplete, unverified, or awaiting approval	Exclude from public navigation, search, and generated artifacts; not authoritative
approved	A policy, decision, or controlled document completed approval but is not being used as a release-specific current-behavior claim	Publish only when classification is public; retain approval evidence
current	Content is verified and applies to the named release	Publish only when classification is public
target	Content describes future intent not present in the current release	Publish only when classification is public and an explicit target notice is present
planned	Content records planning intent that is not yet an approved current-product contract	Publish only when classification is public and the page is clearly marked as planning or roadmap context
deprecated	A current capability or procedure is being withdrawn	When public, keep visible with replacement and end date
historical	Content is accurate for a named prior release, event, or decision period	When public, keep version context and prevent use as a current procedure
archived	Content is retained only as historical evidence	Exclude from public navigation, search, and generated artifacts; preserve version context in controlled storage

“Current” means evidence-backed for the page's `applicable_version`; it does not mean timeless.

Classification and status are independent gates. Public distribution requires `classification: public` and an eligible reviewed status. Draft, archived, internal, confidential, and restricted pages must never appear in public navigation or public artifacts.

Lifecycle stages

1. Propose

Define the audience, user or business question, canonical location, owner, classification, and evidence required. Check for an existing page before creating another source of truth.

2. Draft

Inspect the current repository and, where relevant, the running product. Explain purpose, responsibilities, boundaries, failure behavior, and safe next actions. Use synthetic examples only.

3. Review

Obtain reviews proportionate to risk:

- product review for behavior and terminology;
- engineering review for implementation and contracts;
- security/privacy review for sensitive data, secrets, threat boundaries, or reporting;
- operations review for runbooks, recovery, deployment, and rollback;
- language review for public English or German user content.

The author must not be the sole approver of a high-risk policy or runbook.

4. Verify and publish

Resolve review findings, validate links and metadata, confirm the applicable release, and record `last_verified`. Change status to `current` only after the described behavior and required controls are present.

5. Maintain

Review on the scheduled date and whenever a trigger occurs. Small wording corrections do not reset technical verification unless they change meaning.

6. Deprecate or archive

When behavior changes, either update the canonical page or preserve a versioned historical record.

Deprecated pages must identify the supported replacement. Archived pages must retain their version, owner at archival, and reason.

Review triggers

Review a page immediately when any of the following changes:

- a user-visible workflow, label, default, limit, or supported platform;
- a data schema, backup format, migration, or retention rule;
- a connector route, third-party contract, credential flow, or provider model;
- an encryption, origin, allowlist, redaction, or secret-handling control;
- a release gate, deployment path, signing state, or recovery procedure;

- an owner, classification, legal basis, or support boundary;
- a test reveals the documented statement is false or incomplete.

Review cadence

Default maximum review intervals are:

- 90 days for security, privacy, connectors, AI providers, releases, and recovery;
- 180 days for product behavior, user guides, and operational procedures;
- 365 days for stable concepts, glossary entries, and historical decisions.

A page may specify a shorter interval. An expired review date makes the page stale; it does not automatically make the content false.

Emergency corrections

For a safety-critical documentation defect, publish a narrowly scoped correction immediately, label any unverified remainder, and complete normal review as soon as practical. Never delay a warning about possible data loss, credential exposure, or unsafe recovery merely to preserve the ordinary review sequence.

See [Contributing documentation](#) for the working procedure.

Documentation Style Guide

Mandatory language, terminology, structure, and accessibility conventions for all Klar documentation.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Documentation
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Mandatory Resume terminology

Use **Resume** for Klar's named canonical document, workspace, fields, capability, or generated artifact, and at the beginning of a sentence. Use **resume** only for a generic career document in ordinary prose. The canonical English term contains no accent marks.

This rule applies to:

- the Klar interface;
- documentation and help text;
- code comments, identifiers intended for people, test descriptions, and fixtures;
- generated document labels and filenames;
- release notes, issues, and support material.

Do not introduce accented spellings as variants, quotations, or search aliases in Klar-owned content. German user content may use the established German word **Lebenslauf**. Code identifiers remain resume unless a language-specific UI label is required.

Voice and intent

Write in clear, direct language. Lead with the outcome or decision, then explain constraints and procedure. Prefer “Klar stores the workspace in this browser” over abstract or promotional language.

Use:

- **you** for user instructions;
- **Klar** for product behavior;
- an explicit role, such as “release owner,” for organizational duties;
- **must** for mandatory requirements, **should** for strong recommendations, and **may** for optional actions.

Avoid claims such as “secure,” “private,” “complete,” “accurate,” or “AI-powered” without stating the relevant boundary and evidence.

Current versus target language

Current pages use present tense only for verified behavior. Approved future behavior belongs on a page with `status: target`; planning intent that is not a current product contract uses `status: planned`. Both should use phrases such as “the target is” or “the planned capability would.” Never hide a missing implementation behind future-tense wording inside a current user guide.

Page structure

Each page should answer, as applicable:

- 1 What is the outcome or purpose?
- 2 Who is the page for?
- 3 What is in scope and out of scope?
- 4 What must be true before starting?
- 5 What is the current procedure or contract?
- 6 What can fail, and what remains safe?
- 7 What limitations or human checks apply?
- 8 What related page owns the next level of detail?

Use one # heading for the page title, meaningful ## sections, and short paragraphs. Use tables for exact comparisons and lists for independent items. Do not use a table when a short sentence is clearer.

Product and interface terms

Use the interface's established capitalization:

- **Klar**
- **Career Roles** when naming the product mode; “career roles” in general prose
- **Flexible Work** when naming the product mode
- **Resume** according to the mandatory rule above
- **Tracker, Dashboard, Settings, and Application Packet** when naming a surface
- **Groq, Adzuna, Bundesagentur für Arbeit, Cloudflare Worker, and GitHub Pages** as proper names

Do not call local vocabulary ranking a neural embedding model. Do not describe a ranking score as a hiring probability.

Instructions and safety

- Number steps when order matters.
- State destructive consequences before the action.
- Name what remains unchanged after a failed action.
- Put prerequisites before the first step.
- Give the recovery route for data loss, lockout, connector failure, and export failure.

- Never ask a user to paste a real resume, application document, credential, private path, or unredacted diagnostic into a public issue.

Examples, code, and data

Examples must be synthetic. Do not use a real person's contact details, career history, API key, employer-private material, or local path. Code samples must be minimal, runnable for the applicable version, and explicit about placeholders. Explain why a non-obvious command or configuration exists.

Inline code comments should explain intent, constraints, or surprising behavior. They should not restate syntax. Technical reference pages may document exported symbols; user guides should describe outcomes rather than internal function names.

Links and accessibility

Use descriptive link text, such as [Backups and recovery](#), rather than “click here.” Do not rely on color, position, or an icon alone to communicate state. Tables need clear headers. Images need meaningful alternative text when they carry information.

Bilingual content

The current Klar interface supports English and German. Public user guidance should preserve the same meaning in both languages when localized; one language must not make a stronger product or privacy claim than the other. Klar-owned engineering documentation is English-first unless an approved need requires otherwise.

Documentation Governance

Ownership, approval, classification, review, and control model for the Klar knowledge base.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Documentation
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Governance objective

Documentation is part of Klar's product and control surface. Governance ensures that pages have accountable owners, appropriate access, evidence-backed claims, and predictable review rather than becoming an unmanaged collection of notes.

Public-by-default policy

Klar documentation is intended for public, unauthenticated access after review. Product, user, architecture, engineering, data, quality, operations, and governance detail should be classified `public` when it can be published safely. Technical depth, implementation detail, or an audience of developers is not by itself a reason to restrict a page.

Before a page becomes public, its owner and reviewer must confirm that it contains none of the following:

- personally identifiable information or real Resume, application, contact, employer-private, or support-case data;
- passwords, API keys, tokens, recovery material, or any other secret;
- private infrastructure identifiers, private endpoints, non-public environment details, or access instructions;
- active incident evidence, unredacted logs, investigation records, or affected-party information;
- signing keys, signing certificates not already intentionally public, release credentials, or equivalent signing material; or
- actionable detail for an unfixed vulnerability or exploit when publication would increase risk.

Use synthetic examples and bounded public explanations. When an accurate repair record requires excluded detail, keep that record in an appropriately protected system and publish only the safe limitation, impact boundary, remediation status, and responsible reporting route.

Only pages with `classification: public` may enter the public website, navigation, search index, downloadable handbooks, or other public generated artifacts. Drafts, archived pages, and every non-public classification are excluded even if their source files are present beside public content.

Roles

Role	Accountability
Documentation owner	Maintains taxonomy, standards, tooling, and stale-page reporting
Page owner	Keeps a page correct, classified, reviewed, and linked to evidence
Subject-matter reviewer	Verifies product, engineering, security, privacy, legal, or operational meaning
Approver	Accepts the page for its intended audience and risk level
Release owner	Ensures release changes and evidence are reflected before publication
Contributor	Proposes accurate, scoped changes and resolves review findings

One person may hold several roles for low-risk pages. High-risk policy, security, privacy, credential, encryption, deployment, or recovery content requires independent review.

Required metadata

Every page must include:

- title
- description
- section
- order
- audience
- status
- classification
- applicable_version
- owner
- last_verified
- next_review
- tags

A page without an accountable owner or applicable version cannot be current.

Classification

Classification	Intended handling
public	Reviewed and safe for unauthenticated publication, including public-safe implementation and control detail
internal	Non-public working or operational context available only to authorized contributors
confidential	Limited business, security, incident, personal, or legal detail; access granted by need
restricted	Secrets, signing material, active incident evidence, private infrastructure access detail, or similarly high-impact content; never stored in the public KB

Classification is determined by content, not by the hosting location. Public is the default outcome only after review; removing obvious secrets is not sufficient when personal data, private infrastructure, active incident material, signing material, or actionable unfixed exploit detail remains.

Approval matrix

Page type	Minimum approval
Public user guide	Product owner and documentation owner
Technical reference	Engineering owner
Security or privacy page	Security/privacy owner and technical owner
Runbook or recovery procedure	Operations owner and a successful procedure test
Architecture or product decision	Named decision authority
Target or roadmap page	Product owner; status remains target or planned according to decision state
Release evidence	Release owner

Change controls

Documentation changes should be reviewed with the implementation change that makes them necessary. A release should not be declared documented while material user behavior, configuration, migrations, controls, or limitations remain inconsistent with the KB.

Automated checks should reject:

- missing or invalid metadata;
- duplicate canonical slugs;
- broken internal links;
- a past `next_review` date without an explicit exception;
- prohibited secret patterns or personal-data fixtures;

- the prohibited accented English spelling of Resume;
- status: current pages that describe an unreleased target as present;
- public pages that expose or depend on non-public evidence as if readers can access it.

Exceptions

An exception must name the requirement, owner, justification, risk, compensating control, approval, and expiry date. An exception does not silently change the standard. Expired exceptions must be removed, renewed through review, or treated as a documentation defect.

Reporting

The documentation owner should publish a periodic view of coverage, stale pages, unresolved gaps, review debt, and release alignment. See [Document lifecycle](#) and [Contributing documentation](#).

Contributing Documentation

Repository-based workflow and review checklist for creating or changing Klar KB pages.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Documentation
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Before writing

- 1 Identify the reader's question and the outcome the page must enable.
- 2 Search the KB for an existing canonical page. Extend it instead of creating a duplicate source of truth.
- 3 Inspect the current Klar repository directly. Do not reconstruct current behavior from an earlier build guide or roadmap.
- 4 Confirm the release, relevant code, configuration, tests, UI behavior, decisions, and known limitations.
- 5 Identify the page owner, reviewers, classification, and review interval.

Create or change the page

Use modular Markdown under `knowledge-base/content`. The filename becomes the documentation slug, so choose a stable, descriptive lowercase name with hyphens. Internal links use `/docs/<slug>`.

Start with this metadata shape:

Write the smallest complete page that owns the subject. Link to deeper reference rather than copying it. Explain purpose, boundaries, prerequisites, procedure or contract, failure behavior, limitations, and ownership where those dimensions apply.

Evidence and claims

- Verify current behavior in code and the supported interface.

- Run the tests proportionate to the changed claim.
- Use only primary sources for unstable third-party contracts.
- Label approved future content target and planning intent planned; keep both out of current user procedures.
- Record uncertainty as an explicit gap instead of choosing a convenient interpretation.
- Use synthetic examples and the required Resume terminology from the [style guide](#).

Validate locally

Before review, confirm:

- required metadata is present and correctly typed;
- links resolve and headings are unique and ordered;
- Markdown renders without broken tables, code fences, or inaccessible link text;
- the page contains no credentials, real career data, private paths, or unredacted diagnostics;
- English uses Resume/resume without accent marks;
- current and target statements are visibly distinct;
- procedures match the current interface and preserve safety warnings;
- code or commands have been tested for the applicable version.

Review and publication

Request the reviewers defined in [Documentation governance](#). Resolve substantive findings in the page or the product; do not weaken wording merely to hide a mismatch. Set status: current only when evidence and approvals are complete.

The KB source is publicly readable in the Klar repository, but repository access does not grant rights to copy, modify, redistribute, translate, or independently deploy Klar. Review the current [Klar LICENSE](#) before proposing code or documentation reuse. Public documentation contributions and software-use rights are separate questions.

For a release change, the pull request should identify:

- the changed behavior or control;
- affected KB slugs;
- tests or evidence used for verification;
- migration, rollback, and support implications;
- pages intentionally unchanged and why.

Maintenance

The page owner reviews the page by next_review and when a lifecycle trigger occurs. If ownership is lost or verification cannot be completed, change the page to draft, mark the uncertain section, or archive it. Stale authoritative-looking content is worse than a visible gap.

Related guidance: [KB charter](#), [Source of truth](#), and [Document lifecycle](#).

Security and Privacy

1 controlled documents in this section.

Security, Privacy, and Threat Model

Assets, trust boundaries, adversaries, controls, residual risks, privacy behavior, and secure-reporting rules for Klar 2.6.0.1.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Security
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose and scope

This threat model defines what Klar protects, which adversaries and failures are in scope, and where protection ends. It covers the static web application, local browser storage, Cloudflare Worker, third-party sources and AI providers, downloaded artifacts, and the experimental Electron capability.

Klar 2.6's desktop capability is experimental and its binaries are not publicly distributed. Security fixes are supported on current main and the latest available build. The project provides best-effort maintenance and no response-time or paid-support SLA.

Security objectives

- 1 Keep Resume, preferences, applications, and credentials local unless a user explicitly invokes an external action.
- 2 Prevent third-party source content and AI output from becoming trusted code or facts.
- 3 Make network relays finite, allowlisted, bounded, and non-persistent.
- 4 Protect selected local content at rest with authenticated encryption when the vault is enabled.
- 5 Prevent the desktop renderer from gaining general operating-system authority.
- 6 Execute only signed, checksummed, compatible local-model packages.
- 7 Fail safely without deleting reviewed state, fabricating results, or exposing sensitive diagnostics.
- 8 Make release and security claims proportional to dated evidence.

Asset classification

Class	Examples	Required handling
Restricted personal	Canonical Resume, contact data, application writing, notes, job decisions, readable backup	Minimize, encrypt at rest when vault enabled, never log, explicit export/share
Restricted credential	Groq key, Adzuna pair, vault/backup passphrase, model private signing key	Never log or place in prompts; separate credential storage; private signing key never packaged
Confidential configuration	Provider endpoint, Worker deployment details, source controls, release evidence	Share only for operational need; redact support output where it identifies private infrastructure
Internal operational	Content-free connector health, status, safe error category, build/test results	Bounded retention and safe tokens only
Public external	Job listings, employer routes, public source responses	Still untrusted input; validate, sanitize, timestamp, and preserve provenance
Public product	Static application assets and approved public KB pages	Integrity through reviewed build/deployment process

Trust boundaries

The material boundaries are detailed in [System Context and Runtime Topology](#). Security review must treat these crossings explicitly:

- document file into renderer parser;
- renderer into IndexedDB or session storage;
- renderer through Worker to source/provider;
- untrusted source/provider response into domain state;
- renderer across preload IPC into Electron main;
- Electron main into the local runtime and model package;
- local protected state into a downloaded file; and
- redacted diagnostics into a public or private report.

Threats, controls, and residual exposure

Local data disclosure

Threats: copied browser profile, shared device, local inspection, lost device, readable backup, forgotten open session.

Controls: origin-scoped IndexedDB, optional PBKDF2/AES-GCM vault, separate content/credential envelopes, passphrase requirements, lock, standard backup credential exclusion, exact warning for readable export, full wipe.

Residual: vault-disabled data is readable to the local browser profile; vault-enabled data is accessible to Klar while unlocked; operating-system compromise, keylogging, screenshots, and malicious same-origin code are outside the at-rest guarantee. Passphrases are unrecoverable.

Web code or dependency compromise

Threats: XSS, malicious dependency/update, compromised static hosting, service-worker persistence.

Controls: React escaping, constrained rendering, production build review, lockfile, syntax/type/test/build gates, production dependency and build-tool audits, web CSP-related discipline, service worker disabled in Electron, content-free diagnostics.

Residual: the browser vault cannot defend against malicious application code executing after unlock. Dependency audits are advisory-database snapshots, not proof of absence. A compromised release channel can undermine local-first guarantees.

Malicious document, job, or AI content

Threats: parser exploit, oversized content, active HTML, prompt injection, schema confusion, fabricated claim, unsafe application URL.

Controls: client-side bounded extraction, fixed projections, source normalization, content-type/size bounds on Fabric, no upstream JavaScript execution, structured-output validation, evidence gates, deterministic rank, human review, HTTPS destination controls on desktop.

Residual: source descriptions and document parsers are complex untrusted-input surfaces. Prompt injection cannot be solved by instruction text alone. User review remains mandatory.

Credential exposure and provider misuse

Threats: key in source control, logs, URL, prompt, browser persistence, public report, malicious endpoint, quota theft.

Controls: credential helpers, separate credential vault, session-only option, Worker deployment secrets, authorization headers, stripped Adzuna query credentials, atomic pair selection, diagnostics redaction, explicit custom endpoint configuration.

Residual: custom endpoints can observe all data intentionally sent to them. Browser-origin policy is not a complete abuse-control boundary, and provider-side retention, quotas, and account security remain external responsibilities.

Worker SSRF, resource exhaustion, and relay abuse

Threats: relay misuse, unsafe redirect or destination handling, oversized responses, high request volume, and upstream secret leakage.

Controls: finite routes, fixed or allowlisted destinations, HTTPS reconstruction, redirect revalidation, private-network screening, bounded high-risk operations, safe errors, and an explicit production origin policy.

Residual: origin policy alone cannot prevent non-browser misuse; resource and client-level abuse controls are not yet uniform across every relay; and destination validation needs further defense in depth. The fixed host catalog reduces but does not eliminate server-side request-forgery risk.

Source integrity and stale opportunities

Threats: broken parser, changed third-party schema, stale/removed vacancy, aggregator impersonation, duplicate conflict, synthetic test data in production.

Controls: normalization, per-source isolation, provenance, cache expiry/revalidation, circuit breaker, kill switch, official-route preference, diagnostics, honest partial state.

Residual: third-party availability and truth are not controlled by Klar. Candidate connectors currently run before full verification, and deterministic fixtures can back the UI when a Worker URL is absent. See [Known Gaps and Risk Register](#).

Backup tampering or destructive import

Threats: corrupt file, wrong passphrase, edited JSON, malicious oversized/nested rows, accidental workspace replacement.

Controls: format/version validation, SHA-256 digest, AES-GCM authentication for encrypted payloads, supported migration list, validate-before-replace, one atomic transaction.

Residual: backup digest proves integrity relative to the file, not authorship; validation is not a deep schema audit of every nested row. A valid readable backup is a sensitive copy outside Klar.

Desktop privilege escalation and model substitution

Threats: compromised renderer invoking OS capabilities, unsafe navigation, Electron misconfiguration, untrusted runtime/model, package traversal/symlink, loopback access, secret leakage, orphaned process.

Controls: sandbox, Node disabled, context isolation, permission denial, CSP/navigation policy, narrow frozen bridge, duplicated authoritative main validation, exact sender checks, signed manifest/checksums, path and file-type validation, compatible trust key, ephemeral authenticated loopback, allowlisted environment, cleanup, explicit fuses, packaged smoke tests.

Residual: experimental binaries are unsigned/ad hoc and not publicly distributed; a packaging compatibility exception remains under review; diagnostics redaction is bounded pattern matching; Electron/runtime/dependency vulnerabilities remain possible. Normal product generation is not yet routed to the local provider.

Privacy behavior by operation

Operation	Leaves the device?	Notes
Edit, rank, track, save search, generate deterministic export	No by Klar design	Public job retrieval may already have occurred; local state remains on device
Retrieve jobs	Yes	Query/location parameters go to direct sources or Worker/upstream; Resume content is not required for raw retrieval
AI Resume extraction or writing	Yes for configured cloud provider; the experimental local path is separate	Only the bounded prompt projection should be sent; provider terms apply
Worker health	Yes	No career content required
Create backup or document	No until the user moves/shares it	Downloaded copy is outside the vault
Prepare issue report	No automatic submission	Klar creates a redacted preview and opens a destination only after confirmation
Clear Klar data	No	Does not delete downloaded files or provider/source records

Klar currently has no account analytics or central content telemetry. Browser operational events are content-free, local, and capped at 100. Desktop diagnostics are memory-only and capped at 200. Limited telemetry improves privacy but reduces fleet-wide detection and incident reconstruction.

Secure reporting

Suspected authentication, encryption, secret-handling, sandbox escape, code execution, update, model trust, or personal-data exposure issues must use [GitHub private vulnerability reporting](#). Do not use a public issue.

[Public GitHub issues](#) are appropriate for ordinary defects only after replacing names, emails, phone numbers, employer-private text, paths, query strings, identifiers, and documents with synthetic examples. Never attach a real Resume, cover letter, application, key, password, token, private signing key, or unredacted diagnostic archive.

The in-app tool does not submit either report automatically. Follow [Operations Runbooks](#) for triage and containment.

Review triggers

Re-run this threat model for any new origin, remote service, account/sync capability, telemetry event, storage store, secret, file parser, HTML renderer, Worker route/host, background task, IPC channel, runtime flag, model format/trust key, updater, signing method, export type, or provider fallback.

Quality and Operations

3 controlled documents in this section.

Test Strategy and Release Gates

Automated, packaged, human, security, and deployment evidence required to change or release Klar.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Quality Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

Klar's release process combines deterministic automation with evidence that code alone cannot provide. A passing unit suite is necessary but does not establish source coverage, writing quality, ranking usefulness, security absence, binary trust, or production readiness.

Quality principles

- 1 Test the contract at the boundary that enforces it.
- 2 Preserve regression coverage for every fixed defect.
- 3 Exercise invalid, partial, cancelled, stale, and recovery paths.
- 4 Keep tests deterministic; freeze time, source input, model output, and hashes where relevant.
- 5 Separate machine diagnostics from human release evidence.
- 6 Verify built and packaged artifacts, not only source.
- 7 Re-run temporal evidence such as dependency audits, source checks, and signatures for every release.
- 8 A gate with missing or ineligible evidence returns **HOLD**, never an assumed pass.

Automated test layers

Layer	Examples	Primary purpose
Pure domain	normalization, deduplication, taxonomy, evidence, ranking features, hashes, migrations	Determinism and invariant correctness
Persistence/crypto	fake IndexedDB, vault transitions, wrong passphrase, backup round trips	Atomicity, authentication, compatibility, retention
Source/session	source parsers, registry, Worker allowlist, retry/deadline, partial batches, diagnostics	Isolation, safety, honest coverage
AI consumers	structured response recovery, invalid schemas, quota/budget, evidence, locale	Fail-closed generated-state behavior
Application/export	packet provenance, interruption, DOCX semantics, Word compatibility, ZIP readiness	Artifact integrity and recovery
Desktop	IPC validation, package installer, runtime lifecycle, recovery, security, issue redaction	Privilege and local-runtime boundaries
Release tooling	human-gate validators, model benchmark schemas, bundle check, notebook freshness	Evidence cannot be forged by the wrong evidence class
Packaged QA	Electron smoke and fuse verification	Final archive preserves runtime and security configuration

The repository currently has 66 `test/*.test.ts` files. `scripts/run-tests.mjs` sorts filenames and executes each in a fresh Node process using `tsx`; any non-zero result stops the suite.

Complete automated gate

`npm run qa` executes:

- 1 service-worker syntax check;
- 2 theme-bootstrap syntax check;
- 3 generated Kaggle notebook freshness check;
- 4 application TypeScript check;
- 5 generated Worker binding type check;
- 6 Worker TypeScript check;
- 7 all repository tests;
- 8 production web build;
- 9 bundle-size check;
- 10 desktop-relative renderer build;
- 11 bundle-size check again; and
- 12 Cloudflare Worker dry-run deployment.

The command creates or replaces generated build output. Run it from a worktree where that write is expected.

Bundle ceilings

The generated `dist/assets` gate fails when any uncompressed output exceeds:

Measurement	Ceiling
Largest application JavaScript file, excluding PDF worker	1,000,000 bytes
Total application JavaScript, excluding PDF worker	2,300,000 bytes
Largest stylesheet	100,000 bytes
PDF worker	1,500,000 bytes

These are byte budgets, not Core Web Vitals, memory, accessibility, or runtime-interaction proof.

Web CI and deployment

The “Verify and deploy Klar” workflow runs on pull requests and pushes to `main` using Node 22 and `npm ci`. It performs syntax, notebook, app/Worker type, full test, production build, bundle, and Worker dry-run checks. `VITE_WORKER_URL` is injected from repository configuration.

Pull requests verify but do not deploy. A non-pull-request run uploads the exact verified `dist` and the dependent deploy job publishes it to GitHub Pages. Concurrency cancels an older run for the same reference. A failed verify job must never be bypassed by manually publishing a different local build.

The Worker is deployed/configured separately. A passing Worker dry run does not prove live secrets, origins, routes, or third-party sources.

Desktop preview CI

Desktop-relevant pull requests and manual dispatch run a matrix for macOS 15 arm64 and Windows 2025 x64. Each job:

- 1 installs with `npm ci`;
- 2 runs `npm run qa`;
- 3 runs the production dependency audit;
- 4 installs the pinned platform runtime;
- 5 creates an ephemeral CI model-trust identity;
- 6 builds the unsigned/ad-hoc preview archive;
- 7 smoke-tests the unpacked application and verifies fuses; and
- 8 hashes the exact archive.

The workflow deliberately does not publish the unsigned binaries. Its ephemeral key proves mechanics, not production model trust.

Production desktop distribution additionally requires the full build-tool audit, real trust/signing material, macOS notarization, Windows signing, final artifact hashes, packaged smoke/fuse evidence, provenance, and explicit release approval. Current public desktop distribution remains **HOLD**.

Dependency security gates

- `npm run audit:production` checks production dependencies at high severity.
- `npm run audit:build-tools` checks the complete lockfile at high severity.

Audit results are temporal. The repository records a green result for the lockfile on 10 August 2026, but every release must run both again against the current advisory database. A non-zero result returns distribution to **HOLD** until fixed or explicitly risk-reviewed; a dated prior result cannot be reused.

Ranking human gate

Automated ranking tests prove mechanics, not usefulness. Eligible release evidence requires:

- at least six scenarios and six job families;
- English and German coverage;
- at least three candidate seniority bands;
- at least 50 frozen jobs and one comparison pair per scenario;
- a final held-out corpus with requirements and hashes;
- exactly two distinct professionally bilingual human reviewers working independently, blind to model scores/order and to each other; and
- human adjudication of every disagreement and required comparator analysis.

The frozen decision gates are:

Gate	Threshold
NDCG@10 improvement	At least 20% relative to frozen 2.5.5
Human pairwise agreement	At least 80% with a decisive pair
Known hard-constraint violations in top 10	At most 2%
Precision	Current P@5 and P@10 no worse than 2.5.5
Severe seniority mismatch	Current top-10 rate no worse than 2.5.5
Determinism	Repeat, hash, and reversed-input order stable
Slices	No language, family, or seniority slice failure hidden by an average

Synthetic, test, non-human, non-blind, incomplete, or unadjudicated evidence is development-only and returns **HOLD**. The current human ranking release gate is **HOLD**.

Writing human gate

Eligible writing evidence requires at least 10 samples in each combination of recruiter message/cover letter and English/German: at least 40 outputs total. It uses a held-out, separation-audited corpus, two distinct professionally bilingual independent human reviewers blind to generator/variant and each other, complete claim inventories, and human adjudication.

Frozen gates are:

- recruiter messages rated send-as-is or light-edit at least 90% overall;
- at least 90% separately in English and German;
- exactly zero adjudicated unsupported claims;
- exactly zero unresolved/unclear claims; and
- 100% complete final claim inventory.

The schema also collects human-likeness, reason clarity, request/channel fit, and German formality for analysis, but these are not separate pass thresholds in the current gate implementation. The current human writing release gate is **HOLD**.

Model benchmark evidence

The local-model benchmark validates package compatibility, base/precision/writer capabilities, strict structure, evidence behavior, latency/token/memory measurements, cancellation, crash recovery, and artifact provenance. Synthetic bilingual fixtures are diagnostics only and do not replace writing/ranking human evidence.

Benchmark runner exit semantics are:

- 0: all required evidence passed;
- 1: a required check failed;
- 2: required evidence is on hold or was not run; and
- 64: invalid command or configuration.

Missing precision/writer adapter artifacts therefore produce **HOLD**, not an inferred quality pass.

Manual and external gates

Before a public release, owners must also confirm:

- current supported browser and device journeys;
- English and German critical flows;
- restore and destructive wipe warnings;
- live Worker origin/secrets and health;
- source verification and fallbacks using permitted evidence;
- accessibility and keyboard behavior proportionate to the changed surfaces;
- actual generated-document inspection in supported office/PDF readers;
- privacy/security disclosure accuracy;
- signing/notarization where applicable; and
- all high-priority open risks have an accepted disposition.

Dated authoring verification

During this KB baseline review on 10 August 2026, application type-check, Worker type-check, and the full 66-file regression suite completed successfully against the then-current workspace. The production build, bundle gate, Worker dry run, dependency audits, packaged desktop checks, live sources, and human gates were not rerun as part of the documentation-only task. This is supporting authoring evidence, not release approval.

Operational response and rollback are in [Operations Runbooks](#). Open qualifications are in [Known Gaps and Risk Register](#).

Operations Runbooks

Actionable diagnosis, containment, recovery, verification, and escalation procedures for Klar service and data incidents.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Operations
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-11-10

Purpose

These runbooks turn a symptom into safe containment and a verifiable outcome. Klar has no central user-content backend, so operations focus on the static release, Cloudflare Worker, third-party source/provider boundaries, local recovery, and desktop preview artifacts.

Universal incident procedure

For every incident:

- 1 **Classify impact:** privacy/security, data loss, source integrity, generated-claim integrity, availability, or release/distribution.
- 2 **Preserve minimum evidence:** release, commit, platform, time, safe error category, source/connector ID, and reproduction with synthetic data. Do not collect a real Resume, application, key, password, prompt, or unredacted backup.
- 3 **Contain:** disable the affected connector/feature, stop distribution, roll back the static release, or stop the desktop runtime as appropriate.
- 4 **Protect local state:** do not recommend clearing data before offering and verifying a safe backup when the workspace is accessible.
- 5 **Diagnose against the current code/configuration:** do not rely on an old roadmap or build guide.
- 6 **Recover and verify:** use the success criteria in the relevant runbook and run proportionate regression/release gates.
- 7 **Communicate honestly:** distinguish no results from insufficient coverage, deterministic from AI output, candidate from verified connector, and preview from production.
- 8 **Record:** root cause, affected versions, decision, tests, rollback, documentation change, and residual risk without personal content.

Suspected security or personal-data exposure uses private vulnerability reporting under [Security, Privacy, and Threat Model](#).

Static web release failure

Trigger: blank page, asset 404, broken `/klar/` routing, wrong `version.json`, widespread regression after Pages deployment.

Containment: stop further merges/deployments. Identify the last known verified main deployment. Do not publish a locally built `dist` outside the workflow.

Diagnosis: check Pages workflow result, uploaded artifact, Vite base, `version.json`, service-worker cache behavior, browser console with synthetic state, and whether the fault is cached or in the deployed artifact.

Recovery: fix forward through a reviewed commit or revert the specific release commit through normal version control, then allow the verify/deploy workflow to build and deploy the exact source. If stale service-worker state is involved, update cache/version handling rather than instructing destructive data wipes.

Success: new workflow is green; fresh and previously installed clients load; local IndexedDB state remains; version metadata is correct; critical Career/Flexible/vault routes pass smoke review.

Worker outage or misconfiguration

Trigger: health failure, widespread BA/Adzuna/Fabric/Groq relay errors, CORS rejection, missing secrets, unexpected 403/404/5xx.

Containment: preserve local deterministic functionality. If a route could expose secrets or become an open relay, disable/rollback Worker deployment immediately. If only one source is affected, use its kill switch or remove it from the source plan rather than disabling unrelated routes.

Diagnosis: check `/health`, Worker deployment/version, configured origin list, secret presence without revealing values, route/method, upstream status class, response-size/content-type rejection, and content-free logs/traces.

Recovery: restore the last verified Worker or deploy a reviewed fix after Worker types, security tests, generated binding check, and dry run. Confirm the Pages build points to the intended Worker URL.

Success: allowed production origin succeeds; disallowed origin remains rejected; absent/invalid credential fails safely; route bounds and no-store headers remain; client shows partial states honestly during any remaining source outage.

Connector breakage or source drift

Trigger: parse failures, zero rows inconsistent with official inventory, stale links, blocked/CAPTCHA response, timeout surge, schema failure, duplicate explosion.

Containment: kill the connector locally/configurably, open its circuit, or remove it from the enabled plan. Do not bypass access controls, CAPTCHA, robots restrictions, or explicit blocking. Keep fallback and other sources active.

Diagnosis: verify official ownership and permitted route; inspect a content-redacted structural sample; compare registry host/path/parser version and Worker allowlist; check pagination, content type, expiry, fallback, health, and last verified time.

Recovery: update parser/contract and safe fixtures, or downgrade support to the honest official route plus baseline aggregator fallback. Candidate connectors remain candidate until the full verification gate is complete.

Success: inventory and zero-inventory fixtures pass; malformed/timeout/removal are isolated; provenance is correct; fallback is usable; session remains under 60 seconds; UI does not claim complete direct coverage.

Synthetic Flexible Work data in a live build

Trigger: usingFixtures=true, recognizable fixture rows, or no configured Worker on a production deployment.

Containment: stop describing results as live. Disable affected production entry point or restore the correctly configured build. Preserve fixture mode for controlled development only.

Diagnosis: confirm baked VITE_WORKER_URL, static artifact provenance, Worker health, and environment separation.

Recovery: rebuild through CI with the reviewed Worker URL and add/verify a release assertion that production cannot silently present fixture content as live.

Success: production result provenance is real and verifiable; offline/development fixture state is unmistakably labelled; no synthetic opportunity is cached as live user data.

AI provider, credential, quota, or schema failure

Trigger: unauthorized response, quota/rate error, model retired, empty/invalid structured output, timeout, malformed generated content.

Containment: stop automatic retry loops; preserve deterministic result and last reviewed packet artifact. Never ask for a key in a public report or log it.

Diagnosis: identify engine, endpoint class, model identifier, relay/direct path, credential retention mode, error category, budget state, and consumer validation failure using synthetic input.

Recovery: correct the credential/endpoint/model selection, wait for bounded quota reset, or deploy a compatibility fix with one bounded retry. A new provider/model requires full prompt, schema, evidence, privacy, and release review.

Success: valid structured response passes the capability consumer; invalid responses fail closed; no false 0/100 score or empty artifact is cached; prior reviewed content remains intact.

Vault cannot unlock

Trigger: incorrect-passphrase/authentication failure or unreadable vault.

Containment: do not clear the database, disable encryption, or repeatedly edit the ciphertext. Confirm the person is in the same browser profile/origin and is not entering a backup password instead of the vault passphrase.

Diagnosis: distinguish wrong passphrase, origin/profile mismatch, corrupted local storage, and unsupported envelope version. Collect only error category/version-not ciphertext or personal data-in a report.

Recovery: use the correct passphrase or import a known-good complete encrypted backup with its correct password. Klar cannot recover or reset a forgotten vault passphrase. If neither exists, be explicit that protected content is unrecoverable.

Success: authentication succeeds, content loads, sensitive plaintext stores remain empty while vault is enabled, and a fresh complete encrypted backup can be verified.

Backup restore

Trigger: new device/profile, local corruption, or deliberate rollback.

Precondition: preserve the current workspace with a new backup if it can be opened. Work on a copy of the candidate backup. Confirm file mode, format, version, and password availability.

Procedure: use the in-app import; review its validation result; provide the backup password only in the local UI; allow decrypt/authenticate/migrate checks to finish before replacement; reload after the atomic commit.

Do not: edit encrypted JSON, paste it into support, treat the SHA-256 digest as author authentication, or import an unknown file merely because it parses.

Success: canonical Resume, preferences, searches, packets, tracker, vault state, and relevant caches/counts reconcile; credentials are present only when the selected backup mode included them; wrong password/corrupt copy still fails without modifying the workspace.

Export or document failure

Trigger: blank/corrupt DOCX/PDF/ZIP, missing content, unsupported claim warning, wrong language, stale provenance, mobile download failure.

Containment: do not send the document. Preserve packet and reviewed decisions; record a content-free `export_failure` event if the product path does so.

Diagnosis: check packet kind/job snapshot, language workspace, Resume presence, accepted blocked changes, cover-letter provenance, generator/format version, browser download support, and semantic document checks.

Recovery: resolve evidence/readiness blockers, regenerate through the current exporter, and test the exact artifact in supported readers. Do not manually relabel historical provenance.

Success: selectable text, correct language and facts, current provenance, no placeholders or blocked claims, expected ZIP contents, and successful user-controlled download.

Desktop runtime or package failure

Trigger: package rejected, runtime fails warmup, generation hangs, cancellation fails, process crashes, signature/compatibility error, orphaned process.

Containment: stop the runtime and distribution of the affected package. Do not weaken signature, checksum, path, compatibility, sandbox, IPC, CSP, or fuse checks to make a package run.

Diagnosis: use redacted in-memory diagnostics; confirm memory tier, platform, app/runtime/prompt compatibility, trusted public key ID, checksum/signature, runtime provenance, one-request limit, and child-process state. Keep private keys and model paths out of reports.

Recovery: install a correctly signed compatible package or a reviewed runtime; correct manifest/provenance through the packaging pipeline; run lifecycle, cancellation, crash recovery, packaged smoke, and fuse checks.

Success: verified package installs atomically, warmup succeeds on authenticated ephemeral loopback, cancel returns terminal state, close/quit leaves no child process, and the exact archive hash is recorded.

Suspected vulnerability or personal-data exposure

Trigger: secret leak, sandbox escape, code execution, unauthorized network transfer, model-package trust bypass, encryption/authentication flaw, public personal data.

Containment: stop affected deployment/distribution, revoke exposed provider/deployment credentials, rotate trust material when compromise is plausible, preserve minimal private evidence, and avoid public discussion.

Escalation: use GitHub private vulnerability reporting. Coordinate disclosure only after containment and a decision. Public issues must never contain real personal documents, credentials, paths, or unredacted diagnostics.

Success: root cause fixed, credentials/trust roots rotated where necessary, regression and release gates pass, affected claims/docs updated, and disclosure decision recorded privately.

Local data wipe

Trigger: explicit user request to remove Klar's local workspace.

Procedure: explain that wipe clears all 17 Klar IndexedDB stores but not downloaded files, backups, browser sync copies, provider records, or employer submissions. Offer a backup only if the person requests retention. Require the product's explicit destructive confirmation, execute the wipe, and verify onboarding appears with no prior state.

Success: all Klar stores are empty in the active origin/profile and the user understands remaining external copies.

Evidence record template

For any resolved operation, record: incident ID; dates; affected release/commit/platform; non-personal symptom; impact class; containment; root cause; changed code/configuration; checks run; artifact/deployment hash where relevant; recovery result; residual risk; linked KB updates; approver. Store no Resume, application, key, prompt, or source response body in this record.

Known Gaps and Risk Register

Verified current limitations, control gaps, evidence holds, accepted boundaries, and closure criteria for Klar 2.6.0.1.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Engineering
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-09-10

Purpose

This register prevents implemented foundations, roadmap intent, or passing machine tests from being mistaken for completed product or release claims. It records gaps verified during the 2.6.0.1 code review. Priority describes potential product impact, not a formal vulnerability score.

Priority	Meaning
Critical	Immediate credible risk of severe exposure, irreversible loss, or unsafe distribution; stop affected operation
High	Material integrity, privacy, availability, or release-claim risk requiring explicit disposition before broad release
Medium	Bounded control or reliability gap requiring planned correction or documented acceptance
Low	Maintainability, terminology, or evidence clarity issue with limited immediate user impact

No item is closed by prose alone. Closure requires the named implementation/control change, tests, relevant human or live evidence, documentation update, and owner approval.

Open and held items

ID	Priority	Area	Current state and impact	Current control	Closure evidence	Owner	Status
KG-001	High	Source verification	Career acquisition fans out across 47 verified and 162 candidate ATS entries. Source Fabric marks only 3 of 36 enabled definitions verified; candidate status is not an execution gate. Configured coverage can be mistaken for verified support and can add load/failures.	Per-source isolation, bounded concurrency/session, fallbacks, provenance, health	Verification field enforced for production plans or a separately approved candidate channel; full connector release evidence and honest UI counts	Source Engineering	Open
KG-002	High	Flexible Work integrity	When no Worker URL exists, the live Flexible Work hook can return deterministic synthetic fixtures with usingFixtures=true. A misconfigured production build could present test opportunities as live.	Runtime marker and deterministic provenance available to UI/tests	Production build/deployment assertion prevents silent fixture mode; visible non-live label in permitted development/ofline mode; deployment test	Platform Engineering	Open
KG-003	High	Human quality evidence	Human ranking and bilingual writing release gates are HOLD . Automated/synthetic tests cannot prove usefulness, sendability, or absence of every unsupported claim.	Strict evidence schemas and fail-closed gate tools	Eligible held-out corpora, two independent bilingual human reviews, adjudication, all frozen thresholds passed	Product Quality	Evidence hold

ID	Priority	Area	Current state and impact	Current control	Closure evidence	Owner	Status
KG-004	High	Desktop distribution	Experimental artifacts are unsigned/ad hoc and are not publicly distributed; production signing/notarization is not complete. Precision/writer adapter packages and evidence remain HOLD .	CI does not publish experimental binaries; explicit fuses, trust package checks, smoke/hashe	Production keys and provenance, audits, signed/notarized exact artifacts, smoke/fuse/hash evidence, adapter evidence or scope exclusion, approval	Release Engineering	Evidence hold
KG-005	High	BA continuity	BA integration depends on separate undocumented public-web search and detail contracts that can change without a public compatibility guarantee.	Contract regressions, source isolation, source status, other sources/fallback	Supported durable interface or monitored contract with prompt kill/fallback and current live evidence	Source Engineering	Open/external dependency
KG-006	Medium	Local-model product path	Provider-neutral cloud/local registry and desktop runtime work in tests/panel, but normal Resume and application-generation call sites still use the existing cloud-compatible client. On-device generation is not a complete user workflow.	Documentation and explicit panel boundary; no silent provider fallback	Production capability routing through explicit provider selection, privacy disclosure, end-to-end evidence/export tests, model/human gates	AI Engineering	Open
KG-007	Medium	Worker resource and abuse controls	Resource limits and identity-aware abuse controls are not yet uniform across every public relay. Browser-origin policy alone is not an abuse-control boundary.	Fixed routes/hosts, provider credentials/quotas, bounded high-risk paths, production origin allowlist	Bounded request/response/timeout policy on every route plus appropriate rate/session abuse controls and load/security tests	Platform Security	Open

ID	Priority	Area	Current state and impact	Current control	Closure evidence	Owner	Status
KG-008	Medium	Worker request-forgery defense	Destination validation needs additional defense in depth across address formats, name resolution, and redirect handling.	Fixed host/path allowlist, HTTPS reconstruction, redirect revalidation, public-network Worker setting	Reviewed destination-validation strategy with comprehensive private-network and redirect security tests	Platform Security	Open
KG-009	Medium	Backup trust	SHA-256 backup integrity is not author authentication, and import validation does not deeply validate every nested row. A self-consistent edited or malicious file can remain risky.	Format/version checks, encrypted AES-GCM authentication, primary structural validation, atomic replacement	Authenticated/signed origin option where required; comprehensive schema/size/depth validation and hostile-import tests	Data Engineering	Open
KG-010	Medium	Flexible pagination	Roadmap promises frozen visible page assignments. Current session keeps stable append order and in-place merges but chunks the evolving ordered array rather than persisting an immutable page ledger.	No intentional re-sort after publish; duplicate enrichment in place; tests	Explicit published-page assignment state and regression tests for late high-score/duplicate batches, filters, back navigation	Discovery Engineering	Open

ID	Priority	Area	Current state and impact	Current control	Closure evidence	Owner	Status
KG-011	Medium	Connector recovery	Breaker opens after four failures for five minutes, but persisted health has no distinct low-frequency canary state; post-cooldown requests are simply eligible. Fallback success can obscure direct-path failure in aggregate observation.	Cooldown, local kill switch, health ledger, fallbacks	Explicit half-open/canary state and metrics separating direct from fallback success	Source Engineering	Open
KG-012	Medium	Operational control	Connector kill switches and feature flags are client-local/static; there is no centrally managed remote operator switch that can disable a bad connector across installed clients without a new deployment or user-local state.	Per-client health/kill state and redeployable configuration	Authenticated privacy-safe remote control or an explicitly accepted static-operation model with tested emergency rollout	Operations	Open
KG-013	Medium	Accessibility/performance evidence	The automated release gate enforces bundle bytes and many UI regressions, but does not run a comprehensive browser accessibility scan or Core Web Vitals/Lighthouse budget.	Semantic UI tests, render tests, manual review, byte ceilings	Defined supported-browser accessibility and performance budgets with deterministic CI and manual assistive-technology evidence	Quality Engineering	Open

ID	Priority	Area	Current state and impact	Current control	Closure evidence	Owner	Status
KG-014	Medium	Observability	Browser events are local/content-free and desktop diagnostics are bounded/in-memory. This protects privacy but limits fleet-wide source/runtime incident detection; regex redaction is not formal DLP.	Fixed event schema, safe tokens, 100/200 limits, user preview before report	Approved privacy-preserving aggregate operations design or documented acceptance; adversarial redaction tests and review	Operations and Privacy	Open/accepted tradeoff
KG-015	Low	Ranking provenance	Outer match rows can still say local-v2.3 while their deterministic snapshot says ranking-v2.6.0, which can confuse consumers.	Snapshot includes authoritative ranking/requirements/explanation versions and historical markers	Migrate/deprecate legacy label with cache compatibility tests and documentation	Discovery Engineering	Open
KG-016	Low	Schema documentation drift	src/db/databases header says six stores hold all state; schema version 7 defines 17 stores.	Executable schema and this KB are authoritative	Correct the stale comment and add a lightweight schema-catalog consistency check	Data Engineering	Open

Accepted product and threat boundaries

These are not defects unless product scope changes, but every claim must preserve them:

Boundary	Current treatment	Reconsider when
No closed-app searching	Static web/desktop session performs work only while open	Background service, account, notification, or scheduled refresh is proposed
Vault is at-rest protection	Does not protect an unlocked compromised renderer/device	Sync, browser extension, privileged integration, or new active-content path is added
Local wipe is local	Does not delete downloads, backups, employer submissions, or provider retention	Klar introduces remote storage or managed deletion
Source listing is an observation	Klar cannot guarantee vacancy truth, completeness, or continued acceptance	A contractual data source with service guarantees is adopted

Boundary	Current treatment	Reconsider when
AI output is a proposal	Human review remains required; no auto-send	Any automated submission or autonomous decision feature is proposed
Limited telemetry	No central Resume/job/prompt analytics	Fleet operations require new measurement; privacy review must approve the minimum aggregate design

Review cadence and escalation

Review this register monthly during experimental development, before every release candidate, after any incident, and whenever a roadmap item is presented as current. A newly discovered credible Critical item immediately blocks the affected release or operation and follows [Operations Runbooks](#).

Roadmap status is reconciled in [Architecture Decisions and Roadmap](#). Test evidence and HOLD semantics are defined in [Test Strategy and Release Gates](#).

Decisions and Roadmap

1 controlled documents in this section.

Architecture Decisions and Roadmap

Retrospective decision index, current-versus-target rules, and implementation reconciliation for the Klar v2.4-to-v3 roadmap.

STATUS	current
VERSION	2.6.0.1
CLASSIFICATION	public
OWNER	Klar Architecture
LAST VERIFIED	2026-08-10
NEXT REVIEW	2026-09-10

Purpose

This page records consequential choices that shape Klar and reconciles planning intent with verified implementation. Most entries are retrospective summaries derived from code, tests, configuration, and the 23 July 2026 “Klar Revised Roadmap - v2.4 to v3.” They are an index for governance, not a claim that separately approved ADR documents already exist.

Decision status model

Status	Meaning
Accepted/current	Decision is embodied in 2.6.0.1 code and remains governing
Accepted/partial	Direction is accepted; some required controls or integrations are incomplete
Target	Planned direction not yet authoritative product behavior
Historical	Describes a superseded implementation or baseline
Superseded	Replaced by a later named decision

Decision index

ID	Decision	Why	Consequence	Status	Primary evidence
ADR-001	Local-first workspace with no Klar account backend	Minimize centralized personal-data custody and keep the tool usable without an account	IndexedDB is authoritative; backup/export provide portability; no remote sync or closed-app work	Accepted/current	src/db, src/storage, application topology
ADR-002	Optional authenticated vault with separate content and credential envelopes	Protect selected at-rest data while allowing standard backups to exclude credentials	Passphrase is unrecoverable; settings and operational caches remain outside content ciphertext	Accepted/current	src/crypto, src/backup
ADR-003	Canonical Resume is the sole career-fact authority	Prevent drift between rich Resume and thin Profile models	Profile is derived; schema change requires coordinated migration and snapshot	Accepted/current	src/resume, Dexie v5 migration
ADR-004	One product, two first-class discovery modes	Career and Flexible Work share a workspace but have materially different inputs	Career is Resume-aware; Flexible Work is Resume-free and owns location/preferences	Accepted/current	src/App.tsx, onboarding and mode routing
ADR-005	Source Fabric, not a universal scraper	Source durability and permission vary by employer and platform	Finite connector contracts, allowlists, provenance, fallbacks, health, and kill switches	Accepted/partial	src/flexible/connectors, Worker Fabric route; verification gaps remain
ADR-006	Bounded progressive Flexible Work sessions	One slow or broken source must not block useful results	10-15-second attempts, two retries, 60-second hard deadline, independent batches, terminal states	Accepted/partial	Flexible session policy and tests; immutable page ledger/canary incomplete
ADR-007	Deterministic-first Career ranking	Ranking must be reproducible, explainable, and usable without provider tokens	AI attention cannot overwrite eligibility, posting confidence, or deterministic order	Accepted/current	src/match, ranking snapshots and tests
ADR-008	AI returns evidence-bound proposals, never automatic facts	Fluent generation can fabricate or overstate	Consumer schemas, evidence links, accept/reject/edit, repeated export gates, human review	Accepted/current	src/llm, src/application, packets/export
ADR-009	Worker is a finite allowlisted relay, not an application backend	Browser CORS/secrets require mediation without accepting arbitrary proxy risk	Fixed routes/hosts, bounded Fabric/Groq, no workspace persistence; abuse/SSRF residuals remain	Accepted/partial	worker/src, Wrangler config

ID	Decision	Why	Consequence	Status	Primary evidence
ADR-010	Desktop privilege separation and signed model-package trust	Local inference needs process/file authority that the renderer must not inherit	Narrow IPC, main validation, sandbox/CSP, ephemeral loopback, Ed25519 manifests, explicit fuses	Accepted/partial	desktop/, package and lifecycle tests; production signing remains HOLD
ADR-011	Explicit provider selection with no silent cloud/local fallback	Privacy and reproducibility require knowing where generation occurs	Provider-neutral registry fails rather than switching provider	Accepted/partial	src/ai; normal workflows still use production cloud-compatible client
ADR-012	Artifact-level version and provenance	Packets can contain artifacts generated at different times and contracts	Each Resume/letter/export retains its own format/generator provenance; old data remains historical	Accepted/current	src/packets, src/export, document contracts
ADR-013	Human evidence is a separate release class	Synthetic/model tests cannot establish human usefulness or truthfulness	Ranking/writing gates require independent bilingual humans and adjudication; missing evidence is HOLD	Accepted/current	qa/ranking, qa/writing
ADR-014	Current, target, and historical documentation never collapse	Roadmap prose must not be mistaken for executable reality	Code/config/tests lead; target language is explicit; old build guides are context only	Accepted/current	Source of Truth
ADR-015	Canonical product term is Resume	One spelling reduces search, UI, schema, and documentation inconsistency	All Klar code-facing and documentation language uses Resume/resume; older source wording is not copied	Accepted/current documentation; target v2.7 plan entry	Product direction and Documentation Style Guide

Formal ADR template and workflow

For a new consequential decision, create a record with:

- 1 ID, title, owner, date, and status;
- 2 context and decision pressure;
- 3 options considered, including “do nothing” where meaningful;
- 4 decision and scope;
- 5 security, privacy, data, accessibility, operational, migration, and cost consequences;
- 6 compatibility and rollback;
- 7 measurable acceptance evidence;

- 8 rejected alternatives and why;
- 9 links to implementation, tests, runbooks, risks, and roadmap; and
- 10 supersession rule.

A roadmap item is not an ADR. A shipped implementation can require a retrospective ADR when the choice has long-lived consequences. Updating an ADR does not itself change the product; implementation and verification must follow normal review.

Roadmap source qualification

The available planning source is dated 23 July 2026 and was written against a 2.2 baseline with a revised 2.3 foundation. It is valuable for intent but predates the verified 2.6.0.1 repository. It also predates the canonical Resume terminology rule. Therefore:

- use it to explain target direction and original exit criteria;
- use current code/configuration/tests to say what exists;
- do not mark a release complete merely because the current version number is later; and
- preserve unmet exit gates as partial or target.

Roadmap reconciliation

Roadmap milestone	Original intent	Verified 2.6.0.1 position	Classification
2.4 - Flexible Work and Source Fabric	Resume-free mode, opportunity/provenance model, connector catalog, progressive 60-second sessions, taxonomy, cache/health/fallback, saved searches	Core mode, registry, connectors, session, taxonomy, provenance, cache/health, fallback, and saved searches are implemented. Candidate connectors execute without full release verification; strict frozen-page ledger, canary, and central operator switch are incomplete.	Partially current; unmet gates remain open
2.5 - Application Quality	Evidence-bound Resume tailoring, bilingual artifacts, DOCX/PDF, persistent packets, recovery, Flexible Work lightweight preparation	Canonical evidence proposals, packets, bilingual workspaces, document/ZIP export, provenance, and recovery are current. Human writing evidence remains HOLD, so application quality is not graduated.	Core current; quality graduation HOLD
2.6 - Trust, Reliability, Validation	Explainable ranking, source health/provenance, advanced dedup, backup, budgets, contract/canary tests, student testing without content analytics	Deterministic ranking, source health/provenance, dedup, backup, byte budgets, and extensive contracts exist. Human ranking/writing, full connector verification, live canary model, comprehensive accessibility/performance evidence, broad student testing, and public desktop distribution are incomplete or not evidenced.	Substantially current; graduation incomplete
2.7 - Daily Workspace	Saved Searches 2.0, progressive refresh, inbox, compare, history, pipeline/follow-ups, daily/weekly views, focus mode; no closed-app search claim	Existing saved searches, tracker, dashboard, and mounted active sessions are precursors. The named 2.7 workspace is not current. Canonical Resume terminology is effective in the KB now and is intended for the 2.7 plan.	Target
2.8 - Job Intelligence	Posting confidence, company/compensation/schedule/commute/application-effort research, skill gaps, scorecard	Posting confidence and some deterministic requirement/suitability primitives exist, but the milestone as a whole is not implemented.	Target with current precursors
2.9 - Personalization and Release Candidate	Approved learning, semantic clusters, expanded packets, interviews, reviewed local KB, command/undo, scale/resilience verification	Some packet, interview, local embedding, history, and KB foundations exist. The integrated release-candidate scope and gates are not current.	Target with current precursors
3 - Career and Flexible Work Operating System	Coherent Discover/Decide/Prepare/Interview/Progress operating system with graduation gates	Product modes and several workflow foundations exist. The v3 system and graduation gates are not achieved.	Target

Current-versus-target writing rules

Use these rules in every plan, release note, KB page, UI claim, and issue:

If evidence shows...	Write...	Do not write...
Code/config and proportionate tests implement behavior	"Klar 2.6.0.1 does..." with the applicable limitation	A stronger guarantee than the test or boundary supports
Foundation exists but normal workflow does not use it	"Foundation/preview capability exists; integration is incomplete"	"Klar runs this workflow locally"
Machine tests pass but human gate is absent	"Automated gate passed; human release gate is HOLD"	"Quality is validated"
Registry entry is candidate or fallback-only	"Configured candidate/fallback"	"Verified direct integration"
Roadmap names a future release	"Target for 2.7/2.8/..."	Present-tense support language
Behavior belonged to an earlier release	"Historical in version X"	An undated current instruction
Evidence conflicts or is absent	"Unknown/open gap" and log it	A convenient assumption

Roadmap change control

A milestone can move from target to current only when:

- implementation and migration are merged;
- relevant automated, human, live, security, and packaged gates pass;
- current limitations and external dependencies are disclosed;
- operations and rollback are documented;
- release metadata and user-facing claims match; and
- the risk register has an explicit disposition for remaining High items.

The consolidated current risks are in [Known Gaps and Risk Register](#), and evidence requirements are in [Test Strategy and Release Gates](#).